

Foundations of Machine Learning (AI2000/AI5000): Assignment-2
Released on: 16th September, 2025
Deadline: 29th September, 2025, EoD (11:59PM, IST)
Maximum Marks: 40M

A. Instructions

- Answer all questions. We encourage best coding practices by not penalizing (i.e. you may not get full marks if you make it difficult for us to understand). Hence, use intuitive names for the variables, and comment your code wherever necessary. Use the markdown cells in the notebook for briefly explaining the objective of a code cell or reporting the observations if asked in the question.
- It is expected that you work on these problems individually. If you have any doubts please contact the TA or the instructor no later than 2 days prior to the deadline.
- Use of LLMs are strictly prohibited. If found, you will be heavily penalized in the respective questions.
- You may use built-in implementations only for the basic functions such as `sqrt`, `log`, `linalg.inv`, etc. from libraries such as `numpy`. For plots, you may use `matplotlib` and generate clear plots that are complete and easy to understand. If required, you may use `numpy multi-variate normal` function. Apart from these any other high-level functionalities are required to be implemented by the students.
- You may use numpy's seed function `np.random.seed()` for reproducibility.
- Wherever asked to prove / verify your experiments using theory, please provide a detailed explanation in the Markdown cells section of your .ipynb file.
- **Students are required to submit a Python notebook titled A2.<your-roll-number>.ipynb, together with a PDF document containing all relevant mathematical derivations as specified in the assignment.**

B. Problems

1. Bayesian Regression (10M)

(a) Sequential Bayesian Update (4M)

In this exercise, we move from frequentist regression to Bayesian regression, focusing on sequential updates of the posterior distribution.

1. Generate a dataset of $N = 100$ noisy sinusoidal observations:

$$y = \sin(2\pi x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.1^2), \quad x \sim \mathcal{U}[0, 1].$$

2. The goal is to fit a 25th-order linear regression model with Gaussian basis functions:

$$y(x; \mathbf{w}) = \mathbf{w}^\top \phi(x).$$

where the feature vector is given by

$$\phi(x) = [1, \phi_1(x), \phi_2(x), \dots, \phi_{24}(x)]^\top,$$

with Gaussian basis functions

$$\phi_j(x) = \exp\left(-\frac{(x - \mu_j)^2}{2\sigma^2}\right), \quad j = 1, \dots, 24.$$

The centers μ_j are to be chosen as evenly spaced points in $[0, 1]$, and the width parameter σ can be fixed as a fraction of the spacing between consecutive centers.

Assume a standard normal prior on the weights:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, I).$$

Perform **Bayesian sequential updates** of the posterior distribution as more training points are added. Derive and implement the recursive update equations for the posterior mean and covariance.

3. **Optional:** Visualize the updates to the distribution of any one weight (say, w_5) on a single plot (to avoid overcrowding, you may plot the marginal distribution every 10^{th} update).

(b) MAP Sampling and Curve Fits (3M)

1. Draw a sample $\mathbf{w}^{(s)}$ from the posterior distribution. Using this sample, compute the regression curve $y(x; \mathbf{w}^{(s)})$.
2. Repeat this process 19 times and plot $M = 20$ sampled curves (on the same plot).
3. Compute the average of these sampled curves and compare it visually against the true sinusoidal function.

(c) Posterior Predictive Distribution (3M)

1. In Bayesian linear regression with Gaussian basis functions, the predictive distribution for a new input x_0 is given by

$$p(t_0 | x_0, X, \mathbf{t}) = \mathcal{N}(t_0 | m(x_0), s^2(x_0)),$$

where the predictive mean and variance are:

$$m(x_0) = \boldsymbol{\phi}(x_0)^\top \mathbf{m}_N, \quad s^2(x_0) = \frac{1}{\beta} + \boldsymbol{\phi}(x_0)^\top \mathbf{S}_N \boldsymbol{\phi}(x_0).$$

The notation is as discussed in class. For 3 randomly chosen inputs $x \in X$ (which are uniformly and randomly sampled from the domain; **NOT** from the dataset), plot the predictive distribution $p(t | x)$ as a Gaussian curve. It should show the probability density of t along the y-axis and the possible target values (t itself) along the x-axis. Each plot should have one curve, $p(t | x)$ obtained using dataset of size $N = 100$. Overlay the predictive mean $m(x)$ on each plot. Shade/highlight the region covering 95% credible intervals around the mean (i.e. $m(x) \pm 1.96 s(x)$) in different colours. This should give you a total of 3 plots.

2. For $N = 5, 20$ and 100 , plot $p(t, x)$ (x along the x-axis and t along the y-axis) by shading areas within 95% confidence interval. Overlay the predictive mean $m(x)$ curve and the actual sine curve in each case. Refer to Page 157 (Figure 3.8) of Bishop for expected result. This should give you 3 additional plots.
3. Report your observations from this exercise.

2. Understanding Bias-Variance Trade-off (10M)

In this exercise, we study the bias-variance trade-off in the context of linear regression models with Gaussian basis functions. Note that unlike the previous question, here, we are taking a point estimate of the parameters.

1. Generate $L = 100$ independent datasets, each consisting of $N = 25$ data points sampled from the noisy sinusoidal process

$$y = \sin(2\pi x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.1^2), \quad x \sim \mathcal{U}[0, 1].$$

2. For each dataset, the goal is to fit a 25th-order **regularized** linear regression model with Gaussian basis functions:

$$y(x; \mathbf{w}) = \mathbf{w}^\top \phi(x).$$

where the feature vector is given by

$$\phi(x) = [1, \phi_1(x), \phi_2(x), \dots, \phi_{24}(x)]^\top,$$

with Gaussian basis functions

$$\phi_j(x) = \exp\left(-\frac{(x - \mu_j)^2}{2\sigma^2}\right), \quad j = 1, \dots, 24.$$

The centers μ_j are to be chosen as evenly spaced points in $[0, 1]$, and the width parameter σ can be fixed as a fraction of the spacing between consecutive centers. Use **regularized least squares** (ridge regression) to estimate the parameters:

$$\mathbf{w}^{(\ell)} = \arg \min_{\mathbf{w}} \sum_{i=1}^N (y_i^{(\ell)} - \mathbf{w}^\top \phi(x_i^{(\ell)}))^2 + \lambda \|\mathbf{w}\|^2,$$

for each dataset $\ell = 1, \dots, L$. (You may derive and use the normal equations rather than performing a gradient descent). Choose three different values of the regularization coefficient λ :

$$\lambda_{\text{low}}, \quad \lambda_{\text{mid}}, \quad \lambda_{\text{high}}.$$

For each choice of λ , produce the following plots:

- (i) A plot showing all 100 estimated regression curves $y(x; \mathbf{w}^{(\ell)})$.
- (ii) A plot showing the mean of the estimated regression curves across datasets, together with the original sinusoidal function $\sin(2\pi x)$.

This results in a total of $2 \times 3 = 6$ plots.

3. Using the plots above, comment on how the bias and variance of the fitted models change with the choice of λ .

3. Understanding Generative Models and Discriminant Functions (20M)

In this exercise we try to deeply understand the connection between probabilistic generative models and discriminant functions, both theoretically and computationally.

1. **Synthetic dataset:** Generate a 2D dataset with two classes, generating 250 samples per class (500 samples in total) using `numpy.random`. Ensure that the classes are not trivially separable. Split into training and test sets.
2. **Implement classifiers from scratch:**
 - (a) Gaussian Naive Bayes (GNB)
 - (b) Bernoulli Naive Bayes (BNB)
 - (c) Linear Discriminant Analysis (LDA, least-squares approach)
 - (d) L2 regularization with LDA.

For each classifier:

- Show / Highlight your implementation of priors, parameters, and decision rule in your code.
- Plot the decision boundary on the dataset.

Hint: Please refer to Bishop, C. M. (2006). Pattern Recognition and Machine Learning Springer (mainly Chapter 4) for the base algorithms used to implement the above questions.

3. **Verification:**

For each model, perform the following consistency checks between theory and your implementation:

- (a) **GNB:** Verify one manually computed log-likelihood against the code output.
- (b) **BNB:** Verify class probabilities after binarization for at least one sample.
- (c) **LDA:** Verify that the learned weight vector matches the closed-form least-squares solution.
- (d) **L2 regularization with LDA:** Similar verification as above.

Hint: Please refer to the following wikipedia pages for the verifications, Naive Bayes Classifiers, LDA.

4. **Classification performance:** For each classifier, compute and report:

- Training and test accuracies,
- Computational complexity with explicit $O(\cdot)$ analysis,
- Sensitivity to class imbalance (re-run with $P(\text{Class A}) = 0.9$) .

5. **Effect of Gaussian noise:** Add Gaussian noise with both low variance and high variance to the dataset. Retrain all classifiers.

- For each classifier, run 10 trials with different random noise and plot the distribution of accuracies (example: box - plots).
- Compare how noise affects decision boundaries visually.

6. **Robustness analysis:** Based on your experiments, discuss which classifier is most robust to:

- Gaussian noise,
- Class imbalance,
- Non-linear separability.

Support your conclusions with evidence from your results.